

2. Ricerca in un array

Strutture indicizzate - array

 Collezione di elementi dello stesso tipo, ciascuno dei quali è accessibile in base alla posizione.

Simile al concetto di Memoria centrale

Caratteristiche tipiche

- memorizzati in una posizione contigua in memoria centrale
- accesso mediante indice
- tempo di accesso indipendente della posizione del dato

Limitazione

- struttura statica: non si possono aggiungere nuove posizioni.

Variabili array:

- sono riferimenti `puntatori` all'array

```
B <- A
B[1] <- 0
stampa(A[1])
```

Se: $A = \{5 \mid 3 \mid 2 \mid 4 \mid 7 \mid 6\}$, eseguendo il codice modifico B, ma B è il puntatore all'array A, quindi stampando A[1] verrà stampato 0

Anche nelle funzioni, quando “si passa” un array come argomento, il valore passato è il puntatore a quell'array.

Ricerca in un array

supponendo che gli indici di `A` vadano da $0, \dots, n - 1$

input array `A`, elemento `x`

output indice `i` tale che `A[i] = x`, `-1` se `A` non contiene `x`

Ricerca sequenziale

```
ALGORITMO ricercaSequenziale(array A[0...n - 1], elemento x) → indice
  i ← 0
  WHILE i < n AND A[i] ≠ x DO
    i ← i + 1
  IF i = n THEN
    RETURN -1
  ELSE RETURN i
```

 **lazy evaluation:** se la prima condizione dell' `AND` è falsa, basta quello per falsificare il controllo e quindi in questo caso il ciclo termina. Discorso simile con l'operazione `OR`. *non vale per tutti i linguaggi.*

- n iterazioni
 - se `A[i] != x` costa $O(1) \rightarrow T(n) = \Theta(n)$

Se fossero stringhe il tempo crescerebbe.

Ricerca in un array ordinato in modo crescente

caso base se l'array è vuoto $\rightarrow$ l'elemento non c'è

altrimenti considero metà dell'array.

Due “cursori” all'interno dell'array che delimitano l'area interessante dell'array.

- sx: prima posizione da considerare
- dx: pima pos da non considerare

```
FUNZIONE ricercaRic(Array A, indice sx, indice dx, elemento x) → indice
  IF dx ≤ sx THEN RETURN -1
  ELSE
    // devo prendere la posizione di mezzo
    m ← (sx + dx)/2
    IF x = A[m] THEN RETURN m
    ELSE IF x < A[m] THEN
      RETURN ricercaRic(A, sx, m, x)
    ELSE
      RETURN ricercaRic(A, m + 1, dx, x)

ALGORITMO ricercaBinaria(Array A[0...n - 1], elemento x) → indice
  RETURN ricercaRic(A, 0, n, x)
```

Nota: questo algoritmo prende il nome di ricerca dicotomica

Cosa contiene il *record di attivazione*

RicercaRic(A, 0, 8, 12)

A	sx	dx	x	m
	0	8	12	4

RicercaRic(A, 0, 8, 12)

A	sx	dx	x	m
	0	4	12	2

RicercaRic(A, 3, 4, 12)

A	sx	dx	x	m
	3	4	12	3

1a chiamata

- spazio di ricerca: n elementi

2a chiamata

- spazio di ricerca: $\frac{n}{2}$ elementi

3a chiamata

- spazio di ricerca: $\frac{n}{2^2}$ elementi

...

i-esima chiamata

- spazio di ricerca: $\frac{n}{2^{i-1}}$ elementi

Per arrivare a uno spazio di **1** elemento

$$\begin{aligned} \frac{n}{2^{i-1}} &= 1 \\ n &= 2^{i-1} \\ i &= 1 + \log_2 n \end{aligned}$$

Cerco caso base: **0** elementi → un ulteriore chiamata

$2 + \log_2 n$ chiamate

Spazio:

$\Theta(\log n)$

Tempo:

$\Theta(\log n)$

Serve veramente usare la ricorsione?

Nel nostro algoritmo → **ricorsione in coda**

Molto spesso da questo tipo di ricorsione è facile ricavare un ciclo **WHILE**

ALGORITMO *ricercaBinaria*(Array $A[0...n-1]$, elemento x) → indice

$sx \leftarrow 0$
 $dx \leftarrow n$
 $pos \leftarrow -1$
WHILE $sx < dx$ **AND** $pos = -1$ **DO**
 $m \leftarrow (sx + dx)/2$
 IF $x = A[m]$ **THEN** $pos \leftarrow m$
 ELSE IF $x < A[m]$ **THEN** $dx \leftarrow m$
 ELSE $sx \leftarrow m + 1$
RETURN pos